

1 – [0.25 valores] Indique, justificando, qual a complexidade do código seguinte, em função de N

```
for(int i=0;i<N;i++)  
    for(int j=0;j<1000;j++)  
        for(int k=0;k<j;k++)  
            soma ++;
```

2 – [0.5 valores] Assuma que dispõe de um método `int pesquisa (int m[], int valor)` que efectua uma pesquisa binária. Este método devolve a posição em que o valor procurado se encontra, ou então um valor negativo (-X) caso este não esteja no array indicado. O valor de `abs (X+1)` indica uma posição em que o valor procurado poderia ser inserido para preservar a ordem. **Este método já existe e não precisa de o fazer.** Construa uma função `int maisProximo(int m[], int v)`, com complexidade $O(\log N)$, que identifica um dos valores mais próximo de `v` dentro da matriz ordenada `m`. Soluções com outras ordens de complexidade não são valorizadas.

Por exemplo, se `m={10, 20, 22, 30}`, então `maisProximo(m, -2)==10`, `maisProximo(m, 21)==20` ou então `22`, e `maisProximo(m, 20)==20`.

3 – [0.25 valores] Modifique o protótipo da função `maisProximo`, descrita na alínea anterior, de forma a que passe a aceitar como parâmetros um `List` de quaisquer elementos comparáveis entre si (em vez do array), ajustando também de forma adequada o tipo do parâmetro `v` e o valor de retorno.

4 – [1 valor] Construa uma classe `Saco` à qual podem ser acrescentados números inteiros.

- Crie um método dessa classe que devolve (e remove) o valor cujo resto da divisão por 100 é o menor entre todos os números inseridos. Utilize a classe `PriorityQueue` de forma adequada para resolver este exercício. Soluções que não o façam não são valorizadas.
- Considere que na criação do `Saco`, este pode receber uma lista de *divisores*, que inclui valores inteiros positivos (maiores que zero). Estes **não** são valores para “acrescentar” inicialmente ao `Saco`; são utilizados para um fim diferente. Pretende-se criar um método `List<Integer> obtemMultiplos(Integer x)`, em que `x` é um dos divisores (caso não seja, o método pode devolver `null`), e que devolve uma lista com todos os números acrescentados ao `saco` que são múltiplos de `x`. Este método deve ter complexidade $O(1)$. Soluções que não respeitem este critério não serão valorizadas.

5 – [1 valor] Considere uma árvore binária de pesquisa, implementada através de classes `Árvore` e `Nodo` já existentes, similares às estudadas nas aulas laboratoriais.

- Construa um método da classe `Árvore` que devolve o número de nodos com exactamente dois descendentes.
- Crie um iterador que percorre os elementos da árvore em ordem de nível (primeiro todos os elementos do nível 1 (a raiz), depois todos os elementos do nível 2 (os descendentes da raiz), etc). Implemente o suporte para todas as excepções possíveis. Assuma que a árvore contém métodos para acrescentar e remover nodos. Deve ser possível remover nodos através do iterador sem perturbar a iteração pelos valores. Pode indicar alterações ou acréscimos a métodos que já existam na árvore (*não pode assumir que existe um método na árvore que devolve os nodos ou valores na ordem indicada – adicionalmente, tal método não é necessário ou recomendado para resolver o exercício*).